

Aula Prática 03

Isolamento de testes, esperas confiáveis e múltiplas listas com TDD

Objetivo: evoluir a aplicação sem perder controle de regressão, evidência e commits.

3 horas · 5 partes práticas · checkpoints · quiz final

A aula alterna checkpoints curtos, testes e commits

00:00–00:15	Abertura e verificação do estado da Aula 2
00:15–00:45	Parte 1 · isolamento dos testes funcionais
00:45–01:10	Parte 2 · waits no lugar de sleeps
01:10–01:45	Parte 3 · múltiplos usuários e falha esperada
01:45–02:35	Parte 4 · URLs, views, templates e API REST
02:35–03:00	Parte 5 · modelo List, URL única e fechamento

Antes de codificar, confirme que a base da Aula 2 está funcionando

Isso evita transformar erro de ambiente em falsa falha de TDD.

```
.\.venv\Scripts\Activate.ps1  
python manage.py test lists  
python manage.py test functional_tests  
python manage.py runserver
```

- Abra o projeto no VS Code e confira o interpretador Python da .venv.
- Use dois terminais: um para comandos; outro para o servidor quando necessário.
- Se o Firefox/Selenium travar, feche processos abertos antes de repetir o teste.
- Não avance se os testes da Aula 2 não estiverem em estado conhecido.

Regra da aula: cada vermelho precisa ser classificado como ambiente, falha esperada ou regressão.

Parte 1 - isolamento dos testes funcionais

30 min de trabalho guiado

PARTE 1

O teste funcional deve ser repetível e não deixar dados no banco real.

O que você vai construir

- Pacote `functional_tests` com `__init__.py`.
- Movimento do teste para `functional_tests/tests.py` usando `git mv`.
- Uso de `LiveServerTestCase` no lugar de `unittest` comum.
- Acesso por `self.live_server_url` em vez de `localhost` fixo.

Isolamento reduz interferência entre execuções e melhora a confiança no teste.

Parte 1: execução em dupla

25 min - uma pessoa altera; a outra registra o que cada teste prova.

Faça o teste funcional rodar pelo framework de testes do Django.

- Crie a pasta `functional_tests` e o arquivo `__init__.py`.
- Mova `functional_tests.py` para `functional_tests/tests.py`.
- Troque para `LiveServerTestCase` e use `self.live_server_url`.
- Execute `python manage.py test functional_tests` duas vezes.

Critério de conclusão: o grupo explica por que o banco de teste evita lixo entre execuções.

Parte 2 - waits no lugar de sleeps

25 min de trabalho guiado

PARTE 2

O teste deve esperar a condição certa, não apenas dormir por um tempo fixo.

O que você vai construir

- Constante MAX_WAIT para limitar a espera.
- Método wait_for_row_in_list_table.
- Captura de AssertionError e WebDriverException.
- Remoção dos time.sleep(1) do fluxo principal.

Esperas orientadas por condição reduzem falso erro e suite lenta.

Parte 2: execução em dupla

20 min - uma pessoa implementa; a outra mede o efeito nos testes.

Troque espera cega por espera controlada por condição.

- Renomeie `check_for_row_in_list_table` para `wait_for_row_in_list_table`.
- Adicione loop com limite de tempo e tentativa a cada 0,5 s.
- Atualize as chamadas no teste funcional.
- Execute `python manage.py test functional_tests` e registre o tempo.

Critério de conclusão: a equipe explica por que `sleep` pode causar não determinismo.

Cheque o diagnóstico: isolamento e espera atacam riscos diferentes

Isolamento

“A execução anterior contaminou o próximo teste?”

Banco de teste + LiveServerTestCase · mais confiança

Espera confiável

“A página já chegou ao estado esperado?”

MAX_WAIT + condição observável · menos falso erro

Use os dois: isolamento protege os dados; waits protegem o tempo de resposta da interface.

Parte 3 - múltiplos usuários e falha esperada

35 min de trabalho guiado

PARTE 3

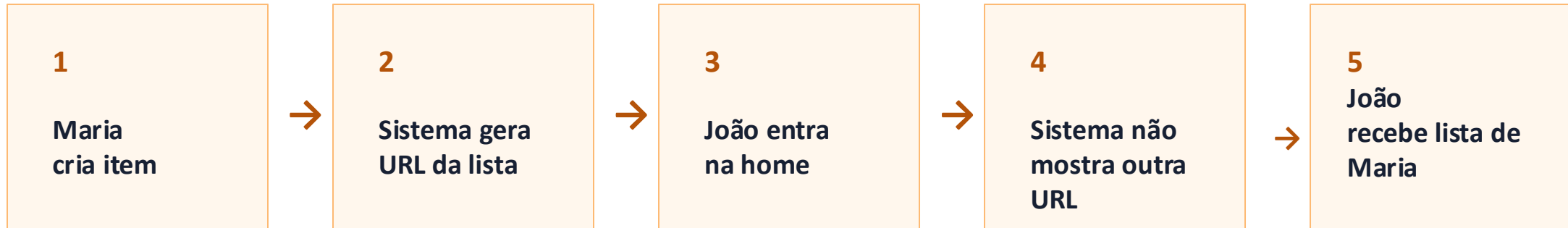
O teste funcional também descreve uma regra de negócio.

O que você vai construir

- Maria cria uma lista e recebe uma URL própria.
- João entra depois e não deve ver a lista de Maria.
- `assertRegex` verifica o padrão `/lists/.+`.
- `assertNotIn` prova isolamento entre usuários.

A falha esperada mostra uma funcionalidade ainda não implementada.

O fluxo desejado passa a ter lista própria para cada usuário



Parte 3: execução em dupla

30 min - uma pessoa codifica; a outra interpreta a falha esperada.

Crie o teste de Maria e João sem tentar corrigir tudo de uma vez.

- Renomeie o teste antigo para `test_can_start_a_list_for_one_user`.
- Adicione `test_multiple_users_can_start_lists_at_different_urls`.
- Execute `python manage.py test functional_tests`.
- Registre a falha do `assertRegex` ou do isolamento de usuários.

Critério de conclusão: a equipe identifica o requisito revelado pela falha.

Parte 4 - URLs, views, templates e API REST

50 min de trabalho guiado

PARTE 4

**A aplicação
começa a
organizar
recursos:
criar lista, ver
lista,
adicionar
item.**

O que você vai construir

- `/lists/new` para criar uma nova lista.
- `/lists/<id>/` para visualizar uma lista.
- `/lists/<id>/add_item` para adicionar item.
- Templates separados para home e lista.

URLs claras ajudam os testes a proteger o comportamento.

Parte 4: execução em dupla

45 min - uma pessoa edita; a outra confere regressão e commits.

Evolua views e templates em pequenos passos.

- Ajuste testes de unidade antes de mudar a view.
- Crie `view_list` e `list.html` quando o teste pedir.
- Simplifique `home.html` para iniciar nova lista.
- Atualize action do form para a URL correta.

Critério de conclusão: testes de unidade passam e o funcional antigo não sofre regressão.

Feche cada avanço com teste, evidência e commit

O commit deve marcar uma decisão técnica verificável.

1. Falha prevista

Execute o teste e leia a mensagem antes de alterar o código.

2. Menor mudança

Implemente apenas o necessário para mudar o resultado do teste.

3. Verificação

Rode unidade e funcional quando a mudança afetar fluxo de usuário.

4. Commit

git status → git add → git commit com mensagem do checkpoint.

Evidências para a entrega no Moodle

A entrega deve mostrar evolução técnica, não apenas código final.

- Execução de `python manage.py test lists e functional_tests`.
- Histórico Git com commits dos checkpoints principais.
- RELATORIO_AULA3.md com falhas esperadas e correções aplicadas.
- Prints ou trechos de terminal que comprovem os resultados.

Dica: uma falha esperada bem explicada vale mais que erro escondido no final.

Quiz final - responda individualmente

Registre as respostas no Moodle ou no instrumento que o professor indicar.

1. O que LiveServerTestCase resolve nesta aula?

- A. Cria CSS automaticamente
- B. Isola banco de teste e sobe servidor para teste funcional
- C. Substitui migrations
- D. Remove a necessidade de Selenium

2. Por que evitar time.sleep fixo?

- A. Porque deixa o teste frágil e lento
- B. Porque impede commits
- C. Porque não funciona com Django
- D. Porque apaga o banco

Quiz final - continue

Responda com base no comportamento observado na prática.

3. O que o teste de Maria e João verifica?

- A. Que todos usam a mesma lista
- B. Que cada usuário pode ter lista e URL próprias
- C. Que o template home.html foi apagado
- D. Que o servidor não precisa iniciar

4. Para que serve /lists/<id>/add_item?

- A. Criar banco de teste
- B. Adicionar item a uma lista existente
- C. Rodar Selenium
- D. Trocar o navegador

5. Qual erro costuma indicar migration ou banco desatualizado?

- A. no such table
- B. assertRegex passou
- C. git status limpo
- D. teste final OK

6. O que uma regressão revela?

- A. Uma mudança quebrou comportamento antes correto
- B. O teste ficou mais bonito
- C. O commit foi enviado
- D. O template não tem HTML

O resultado da aula é evolução controlada da aplicação.

O teste falha → a equipe entende → o código muda → o teste passa → o commit registra.

Próximo passo: usar essa disciplina para testes automatizados mais completos e integração contínua.